

שאלות חזרה נוספות לקראת הבחינה

רועי כנעני

שאלה 1

ראו בעיה 2.2 בספר הלימוד
(CLRS) Introduction to Algorithms, 3rd ed 2009, page 40

האלגוריתם **מיון-בועות** הוא אלגוריתם למיון מערך המכיל n אברים.

הפסאודו-קוד של האלגוריתם:

```
BubbleSort( A , n=length(A) )
1.   for i = 1 to n-1
2.       for j = n downto i+1
3.           if A[j] < A[j-1]
4.               swap( A[j], A[j-1] )
```

א) הדגישו את פעולת האלגוריתם על המערך $A = [5, 1, 4, 2, 8]$
ב) (רשות) הוכיחו (או הסבירו) מדוע האלגוריתם נכון, כלומר ממין את מערך הקלט A בסדר עולה (לא יורד).

ג) הציעו שיפור לאלגוריתם שמפסיק את הסריקה של הלולאות אם בסיום ביצוע של לולאה המערך כבר ממויין.

רמז: השתמשו במשתנה עזר לוגי `swapped` להפסקת הסריקה ברגע שלא בוצעה אף החלפה באיטרציה. ראו גם תאור של האלגוריתם **מיון-בועות** בקישור

https://en.wikipedia.org/wiki/Bubble_sort

ד) נתחו את **זמן הריצה** של האלגוריתם במקרה הגרוע. מהו המקרה הגרוע?

שאלה 3

נתונים מערך $A[1..n]$ של מספרים ממשיים ומספר טבעי m המקיים $2m < n$.
כתבו אלגוריתם הבודק האם קיים ב- A איבר z המקיים את שני התנאים הבאים:
(1) A מכיל לפחות $n - 2m$ איברים קטנים מ- z ;
(2) z מופיע יותר מ- m פעמים ב- A .
זמן הריצה הנדרש של האלגוריתם הינו $\Theta(n)$.

שאלה 4

האם עץ אדום-שחור הוא עץ "מאוזן משקל"?

עץ מאוזן-משקל הוא עץ בינרי T שמקיים את התכונה הבאה:
מספר הצמתים בתת-עץ אחד של השורש הוא לכל היותר פי שניים ממספר הצמתים בתת-עץ השני של השורש.
כלומר, בעץ מאוזן-משקל מתקיים $2 \geq |T_L|/|T_R| \geq 1/2$, כאשר $|T|$ הוא מספר הצמתים בעץ הבינרי T ו- T_L, T_R הם התת-עצים השמאלי והימני של $\text{root}[T]$, בהתאמה.
האם כל עץ אדום-שחור הוא גם עץ מאוזן-משקל?
הוכיחו או תנו דוגמה נגדית.

שאלה 5

הציעו מבנה נתונים S התומך בפעולות הבאות בזמנים הנדרשים (n מציין את מספר האיברים במבנה):

BUILD(S): בניית המבנה S מסדרה של n מפתחות; זמן הריצה: $O(n \cdot \lg n)$;

INSERT(S, k): הכנסת המפתח k לתוך המבנה S ; זמן הריצה: $O(\lg n)$;

DELETE-MAX(S): מחיקת האיבר בעל המפתח המכסימלי מהמבנה S ; זמן הריצה:

$O(\lg n)$

DELETE-OLD(S, t): מחיקת האיבר ה- t הוותיק ביותר מהמבנה S ($1 \leq t \leq n$); זמן הריצה:

$O(\lg n)$

ADD-TO-NEW(S, d): הוספת הערך d לאיבר שנכנס לאחרון למבנה S ; זמן הריצה:

$O(\lg n)$.

שאלה 6

הציעו מבנה נתונים לתחזוקת מטריצה ריבועית $M[n,n]$. המבנה חייב לתמוך בפעולות הבאות בזמנים הנדרשים:

$INIT(M)$: אתחול המטריצה M (כל התאים מקבלים את הערך 0); זמן הריצה $\Theta(n^2)$;

$READ(M, i, j)$: החזרת הערך $M[i, j]$; זמן הריצה $O(1)$;

$UPDATE(M, i, j, v)$: העדכון $M[i, j] \leftarrow v$; זמן הריצה $O(1)$;

$TRANSPOSE(M)$: החלפת האיברים $M[i, j] \leftrightarrow M[j, i]$ לכל (i, j) , $i < j$;

זמן הריצה: $O(1)$;

$UPGRADE(M, d)$: הוספת הערך d לכל איברי המטריצה M ; זמן הריצה $O(1)$;

$SUM(M)$: החזרת סכום כל האיברים של M ; זמן הריצה $O(1)$.

תשובות לשאלות חזרה נוספות

פתרון שאלה 1 (מיון-בועות)

א) נסו בעצמכם.

א+ג) ראו גרסה נוספת (מעט שונה מזו של קורמן) של האלגוריתם **מיון-בועות** בקישור

https://en.wikipedia.org/wiki/Bubble_sort

בפתרון עבור הגרסה של מיון בועות שבויקיפדיה יש מעט שינויים לעומת האלגוריתם שבקורמן: האלגוריתם מניח מערך שמתחיל באינדקס 0.

בסוף כל איטרציה האיבר ה n הגדול ביותר מגיע למקומו הנכון במערך הסופי הממויין

(ולא האיבר ה n הקטן ביותר מגיע למקומו הנכון במערך, כמו באלגוריתם שבספר הלימוד של קורמן).

```
procedure bubbleSort( A[0..(n-1)] )
  n := length(A)
  repeat
    swapped := false
    for i := 1 to n - 1 inclusive do
      if A[i - 1] > A[i] then
        swap(A[i - 1], A[i])
        swapped = true
      end if
    end for
    n := n - 1
  until not swapped
```

Step-by-step example^[edit]

Take an array of numbers "5 1 4 2 8", and sort the array from lowest number to greatest number using bubble sort. In each step, elements written in **bold** are being compared. Three passes will be required;

First Pass

(**5** **1** 4 2 8) → (**1** **5** 4 2 8), Here, algorithm compares the first two elements, and swaps since 5 > 1.

(**1** **5** 4 2 8) → (**1** **4** **5** 2 8), Swap since 5 > 4

(**1** **4** **5** 2 8) → (**1** **4** **2** **5** 8), Swap since 5 > 2

(**1** **4** **2** **5** 8) → (**1** **4** **2** **5** 8), Now, since these elements are already in order (8 > 5), algorithm does not swap them.

Second Pass

(**1** **4** **2** 5 8) → (**1** **4** **2** 5 8)

(**1** **4** **2** 5 8) → (**1** **2** **4** 5 8), Swap since 4 > 2

(**1** **2** **4** 5 8) → (**1** **2** **4** 5 8)

(**1** **2** **4** **5** 8) → (**1** **2** **4** **5** 8)

Now, the array is already sorted, but the algorithm does not know if it is completed. The algorithm needs one **whole** pass without **any** swap to know it is sorted.

Third Pass

(1 2 4 5 8) $\rightarrow$ (1 2 4 5 8)
(1 2 4 5 8) $\rightarrow$ (1 2 4 5 8)
(1 2 4 5 8) $\rightarrow$ (1 2 4 5 8)
(1 2 4 5 8) $\rightarrow$ (1 2 4 5 8)

(ד) ניתוח זמן ריצה של האלגוריתם מיון בועות:

- d. The running time depends on the number of iterations of the **for** loop of lines 2–4. For a given value of i , this loop makes $n - i$ iterations, and i takes on the values $1, 2, \dots, n - 1$. The total number of iterations, therefore, is

$$\begin{aligned}\sum_{i=1}^{n-1} (n - i) &= \sum_{i=1}^{n-1} n - \sum_{i=1}^{n-1} i \\ &= n(n - 1) - \frac{n(n - 1)}{2} \\ &= \frac{n(n - 1)}{2} \\ &= \frac{n^2}{2} - \frac{n}{2}.\end{aligned}$$

Thus, the running time of bubblesort is $\Theta(n^2)$ in all cases. The worst-case running time is the same as that of insertion sort.

- b. **Loop invariant:** At the start of each iteration of the **for** loop of lines 2–4, $A[j] = \min \{A[k] : j \leq k \leq n\}$ and the subarray $A[j..n]$ is a permutation of the values that were in $A[j..n]$ at the time that the loop started.

Initialization: Initially, $j = n$, and the subarray $A[j..n]$ consists of single element $A[n]$. The loop invariant trivially holds.

Maintenance: Consider an iteration for a given value of j . By the loop invariant, $A[j]$ is the smallest value in $A[j..n]$. Lines 3–4 exchange $A[j]$ and $A[j-1]$ if $A[j]$ is less than $A[j-1]$, and so $A[j-1]$ will be the smallest value in $A[j-1..n]$ afterward. Since the only change to the subarray $A[j-1..n]$ is this possible exchange, and the subarray $A[j..n]$ is a permutation of the values that were in $A[j..n]$ at the time that the loop started, we see that $A[j-1..n]$ is a permutation of the values that were in $A[j-1..n]$ at the time that the loop started. Decrementing j for the next iteration maintains the invariant.

Termination: The loop terminates when j reaches i . By the statement of the loop invariant, $A[i] = \min \{A[k] : i \leq k \leq n\}$ and $A[i..n]$ is a permutation of the values that were in $A[i..n]$ at the time that the loop started.

- c. **Loop invariant:** At the start of each iteration of the **for** loop of lines 1–4, the subarray $A[1..i-1]$ consists of the $i-1$ smallest values originally in $A[1..n]$, in sorted order, and $A[i..n]$ consists of the $n-i+1$ remaining values originally in $A[1..n]$.

Initialization: Before the first iteration of the loop, $i = 1$. The subarray $A[1..i-1]$ is empty, and so the loop invariant vacuously holds.

Maintenance: Consider an iteration for a given value of i . By the loop invariant, $A[1..i-1]$ consists of the i smallest values in $A[1..n]$, in sorted order. Part (b) showed that after executing the **for** loop of lines 2–4, $A[i]$ is the smallest value in $A[i..n]$, and so $A[1..i]$ is now the i smallest values originally in $A[1..n]$, in sorted order. Moreover, since the **for** loop of lines 2–4 permutes $A[i..n]$, the subarray $A[i+1..n]$ consists of the $n-i$ remaining values originally in $A[1..n]$.

Termination: The **for** loop of lines 1–4 terminates when $i = n$, so that $i-1 = n-1$. By the statement of the loop invariant, $A[1..i-1]$ is the subarray

$A[1..n-1]$, and it consists of the $n-1$ smallest values originally in $A[1..n]$, in sorted order. The remaining element must be the largest value in $A[1..n]$, and it is in $A[n]$. Therefore, the entire array $A[1..n]$ is sorted.

Note: In the second edition, the **for** loop of lines 1–4 had an upper bound of $A.length$. The last iteration of the outer **for** loop would then result in no iterations of the inner **for** loop of lines 1–4, but the termination argument would simplify: $A[1..i-1]$ would be the entire array $A[1..n]$, which, by the loop invariant, is sorted.

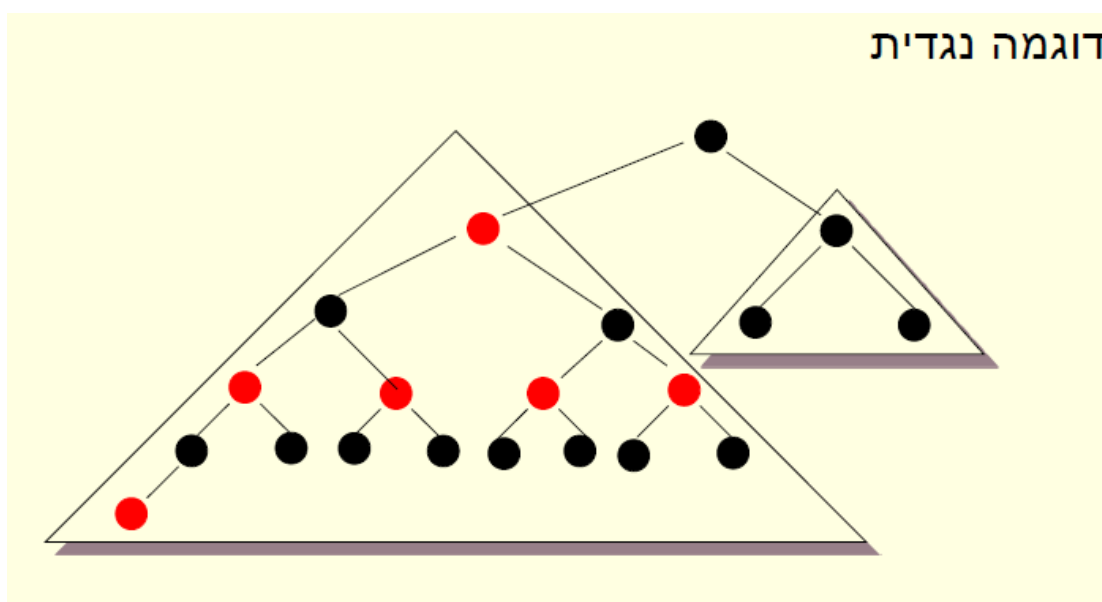
פתרון שאלה 3

המועמדים האפשריים לתנאי הראשון הם כל האיברים שערך המיקום שלהם גדול מ- $n-2m$ כדי שאיבר כלשהו יהיה מועמד לתנאי השני, הוא צריך להיות ערך המיקום ה- $n-m$ או ה- n . אם כך מוצאים את ערך המיקום ה- $n-m$ עם SELECT ועושים עוד מעבר על המערך כדי למנות את מספר המופעים שלו. אם הוא מופיע יותר מ- m פעמים אז סיימנו, אחרת עושים אותו דבר לערך המיקום ה- n . סה"כ קוראים פעמיים ל-SELECT ועושים עוד שני מעברים על המערך לכן זמן הריצה הוא $\Theta(n)$.

פתרון שאלה 4

התשובה היא לא.

דוגמה נגדית



פתרון שאלה 5

מבנה הנתונים S מורכב מעץ אדום-שחור T ומעץ ערכי מיקום W . כל אחד מ- n המפתחות מופיע בעץ T ; זמן ההכנסה שלו משמש במפתח בעץ W . שני הצמתים המקבילים מחוברים ביניהם באמצעות שני מצביעים.

$BUILD(S)$: בניית כל אחד משני העצים אורכת זמן $O(n \cdot \lg n)$.

$INSERT(S, k)$: הכנסת המפתח k לכל אחד משני העצים אורכת זמן $O(\lg n)$; הוספת המצביעים – זמן קבוע.

$DELETE-MAX(S)$: מחפשים את המפתח המכסימלי ב- T באמצעות

$TREE-MAXIMUM(T)$; מחיקת האיבר בעזרת $RB-DELETE(T, \bullet)$ ומחיקת מקבילו בעזרת $RB-DELETE(W, \bullet)$; זמן ריצה $O(\lg n)$.

$DELETE-OLD(S, t)$: מחפשים את המפתח ה- t הקטן ביותר ב- W באמצעות

$OS-SELECT(W, t)$; מחיקת האיבר בעזרת $RB-DELETE(W, \bullet)$ ומחיקת מקבילו בעזרת $RB-DELETE(T, \bullet)$; זמן ריצה $O(\lg n)$.

פתרון שאלה 6

נאתחל מטריצה M ריבועית לאפס, ונחזיק 2 משתנים נוספים: $sumM$ שערך ההתחלתי הוא אפס. $irows$ שערך $true$ אם i הוא אינדקס שורות ו- j אינדקס עמודות, או $false$ אם i הוא אינדקס העמודות ו- j הוא אינדקס השורות.

$INIT(M)$: נאתחל את כל התאים במטריצה דו-ממדית ריבועית M בגודל $n \times n$ לערך אפס (שתי לולאות for מקוננות) בזמן ריצה של $O(n^2)$. נאתחל גם $sumM = 0$, $irows = true$.

$Read(M, i, j)$, $Update(M, i, j, v)$:

בשתי הפונקציות נבצע פשוט גישה ישירה לתא במקום i, j במטריצה בעזרת האינדקסים i, j . $O(1)$.

הערות: 1. נבדוק את ערכו של $irows$. אם הוא $true$ אזי אינדקס הקלט i הוא אינדקס שורות, אחרת אם $irows == false$ פשוט נחליף בין האינדקסים i ו- j בקריאה לפונקציות.
2. בפונקציה $Read(M, i, j)$ נוסיף את הסכום $sumM$ לערך המוחזר מהתא $M[i, j]$.
3. בפונקציה $Update(M, i, j, v)$ לפני העדכון של התא - נוסיף את ההפרש $v - M[i, j]$ לסכום $sumM$. $O(1)$.

$Transpose(M)$: נשנה את הערך הלוגי במשתנה $irows$:

$irows = \text{not}(irows)$

כך שהתפקיד של האינדקס i יהפוך מאינדקס שורות לאינדקס של עמודות (או להיפך). זמן הריצה הוא $O(1)$.

$Sum(M)$: נחזיר את הערך של המשתנה $sumM$ ב- $O(1)$.